

# CSCI 1430 Final Project Report:

## Staff2Sound: A Deep Learning Pipeline for Sheet Music to MIDI

Team name: Peiyuan Li, Yuehuan Qiu, Wanming He, Yifan Sun.

TA name: Joo BumJin. Brown University

### Abstract

*Optical Music Recognition (OMR) aims to extract structured musical information from sheet music images. In this work, we propose **Staff2Sound**—an end-to-end deep learning pipeline that detects musical symbols and converts them into **playable MIDI files**. Our detector is a pre-trained YOLOv8 and later fine-tuned on the DeepScoresV2 Dense dataset. Our method introduces a **pairwise staffline cropping strategy** to improve detection in dense layouts. A **rule-based pitch reconstruction module** maps detected notes to musical pitches using geometric and music-theoretic rules. Staff2Sound significantly improves detection mAP and enables auditory reconstruction from printed scores. All related codes are available at <https://github.com/pli233/staff2sound>*

### 1. Introduction

Optical Music Recognition (OMR) is a longstanding challenge in document analysis and computer vision. It aims to read images of printed or handwritten sheet music. OMR operates on complex 2D layouts with densely packed and often overlapping symbols. This complexity is further exacerbated by highly imbalanced class distributions and the small physical size of key musical elements.

One of the most persistent challenges in OMR is **symbol detection**—a critical precursor to pitch reconstruction and audio generation. Despite recent advancements in deep learning, prior work on the DeepScores dataset demonstrates that general-purpose object detectors such as YOLO or R-CNNs tend to underperform when applied directly to **full-page** sheet music (called music scores in their paper). The *Deep Watershed Detection (DWD)* algorithm, proposed by the original DeepScores team, was explicitly designed for densely populated symbol layouts. However, it still suffers from low accuracy on notehead-related classes, leading to poor mAP scores and limiting its practical utility for applications requiring precise symbol detection.

Beyond detection limitations, the **structure of the DeepScores dataset itself introduces challenges**. Although it

contains bounding boxes for many symbol classes, it does *not provide precise staffline coordinates*. Instead, only large bounding boxes for each entire staff are given, making it difficult to accurately align noteheads to pitches—a requirement for converting visual data into sound. This constraint reduces any postprocessing method’s effectiveness in producing meaningful musical output.

To address these challenges, we propose **Staff2Sound**, an end-to-end deep learning pipeline that detects musical symbols and converts them into **playable MIDI files**. Our approach includes two key innovations:

- A **pairwise staffline cropping strategy** that detects stafflines using a custom filter-based method and segments each sheet into 10-line regions (i.e., pairs of 5-line staves). By reducing the detection context to a smaller, musically meaningful window, we significantly improve note localization accuracy and boost detection mAP, especially for dense layouts.
- A **rule-based pitch reconstruction module** that infers the pitch of detected notes using basic music-theoretic heuristics. This enables accurate symbolic-to-audio conversion and bridges the gap between visual detection and auditory output.

By integrating detection, segmentation, and synthesis, **Staff2Sound** moves beyond conventional OMR pipelines, offering a practical framework that recognizes symbols and brings sheet music to life through sound.

### 2. Related Work

Our project builds upon several key components in OMR.

#### 2.1. Symbol Detection Related Works

Our project relies on the DeepScores [5] dataset, a large-scale collection of annotated sheet music images paired with corresponding symbolic music representations. DeepScores provides high-quality, structured notation data primarily focused on Western classical music, making it ideal for training optical music recognition and music transcription models.

The dataset contains over 300,000 annotated music symbols (notes, clefs, dynamics). It provides pixel-level segmentation for OMR tasks and primarily consists of typeset scores.

A notable baseline on DeepScores is the **DeepWater-shedDetection (DWD)** model [6], developed by the dataset authors. DWD formulates detection as a segmentation task followed by watershed-based post-processing. While effective for symbol classification and segmentation, DWD struggles with dense notehead localization and fails to generate structured musical output such as MIDI, limiting its practical usability in downstream applications.

We employed YOLOv8 [2] to detect musical symbols (e.g., notes, rests, clefs) in sheet music. YOLO is a classic single-stage object detection model. YOLO’s efficiency and accuracy make it well-suited for optical music recognition tasks, enabling rapid detection of notes, rests, clefs, and other notation elements.

## 2.2. MIDI File Related Works

For generating high-quality sheet music from MIDI, we use LilyPond [3], a powerful music engraving tool. LilyPond produces professional-grade notation with precise control over layout, spacing, and stylistic details. Additionally, we used Music21 [1], a symbolic music analysis tool, used for parsing and manipulating MIDI/XML files. We also used MuseScore [4], a library for rendering sheet music in web applications.

## 3. Method

Our proposed pipeline **Staff2Sound** consists of two main components: (1) a **YOLO-based object detection module using pairwise staffline cropping strategy**, which processes raw sheet music images and outputs bounding boxes for musical symbols, and (2) a **rule-based MIDI reconstruction module**, which translates detected bounding boxes into playable musical notes. This two-stage approach enables both high-precision symbol detection and end-to-end auditory output from printed scores.

### 3.1. Bounding box Detection

A key challenge in applying deep-learning object detectors like YOLO to musical scores is the high symbol density and small object size, which lead to poor detection performance on full-page inputs. To address this, we propose a **pairwise cropping strategy** that leverages the repetitive structure of sheet music—specifically, the presence of *staff lines*—to localize and isolate detection regions.

**1. Yolo Symbol Detector.** For our pipeline’s musical symbol detection component, we adopted a YOLOv8 model because of its strong performance in real-time detection tasks. Our model was later fine-tuned on the DeepScoresV2 Dense dataset. You can use other detectors if you prefer.

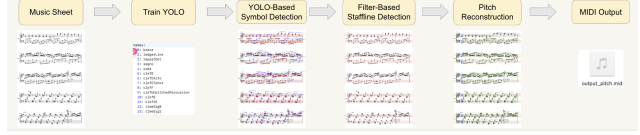


Figure 1. WorkFlow of Staff2Sound

**2. Staffline Detection.** We first convert the input image to grayscale and threshold it at a gray level of 250. To detect horizontal staff lines, we apply a 1D convolution with a wide horizontal kernel (size = 100) and identify rows with strong horizontal responses. We further filter out spurious rows by requiring a minimum proportion (50%) of black pixels per row. Finally, we deduplicate closely spaced lines (less than 4 pixels apart) to avoid redundant detection.

Fig. 2 exemplifies stafflines detected in a sheet music.

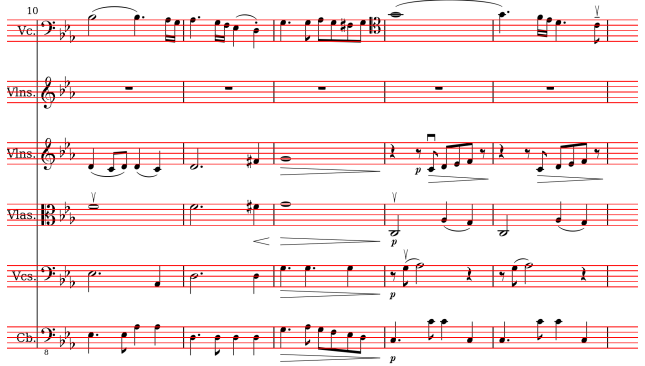


Figure 2. Staffline Detection in Sheet Music

**3. Grouping Stafflines into Pair.** Stafflines are grouped into sets of five, forming one musical staff system. We merge every two such systems (i.e., ten lines in total) into a “staffline pair,” defining the vertical cropping window for YOLO inference.

**4. Region Cropping and Inference.** For each staffline pair, we define a vertical crop region with a 25% margin above and below the extreme stafflines. This localized patch is passed to a trained YOLOv8 model for inference. YOLO returns bounding boxes, class IDs, and confidence scores for each detected symbol in the cropped region.

Fig. 3 exemplifies a staffline pair.

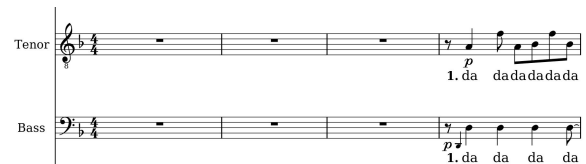


Figure 3. A Staffline Pair Cropped Region

**5. Post-Alignment and Annotation.** Bounding box coordinates are adjusted back to the coordinate system of the original full image by adding the vertical crop offset. We draw the boxes and labels on the original image and save the final annotated image to disk for visualization and downstream processing.

### 3.2. Rule-based Pitch Reconstruction

After extracting bounding boxes and labels from the YOLO model, we perform rule-based pitch reconstruction to convert musical symbols into MIDI events. This process involves several stages:

**Staff Assignment.** Each musical element (e.g., notes, beams, flags, clefs) is assigned to its closest staff by comparing vertical center coordinates. Staves are sorted top-down, and elements are grouped accordingly.

**Note Duration Estimation.** Each notehead or rest’s duration is inferred based on its visual label. For instance, a label containing `Whole`, `Half`, or `Black` is mapped to corresponding MIDI durations (1920, 960, and 480 ticks, respectively). If a note has a flag or is connected to a beam, its duration is shortened accordingly (e.g., 240 ticks for eighth notes). This connection is determined by the distance from the note to the beam or flag based on the threshold of the horizontal width of a note. Fig. 4 exemplifies beams detected.

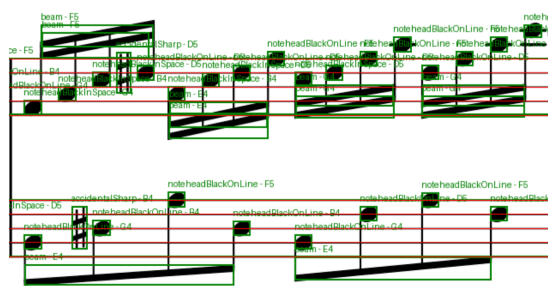


Figure 4. Beams and notes detected and annotated

**Pitch Estimation.** To determine pitch, we first perform filter-based staff line detection to obtain the vertical positions of each staff line, as illustrated in Fig. 5. For each notehead, we compute its vertical distance from the center line of the corresponding staff. This distance is then normalized by half the distance between adjacent staff lines to yield a discrete *staff position index*, effectively quantizing the vertical position into musical intervals. The pitch of each note is determined by mapping the staff position index to a MIDI pitch value. This mapping depends on the clef type (e.g., G-clef

or F-clef), which establishes the reference line for pitch estimation. While MIDI pitch numbers increase uniformly, their musical interpretation varies with tonality. As illustrated in Fig. 6, the mapping from a visual staff position to MIDI pitch is non-isometric. We restrict our pitch reconstruction to the C major scale for simplicity and consistency. Each staff position index is mapped to its corresponding pitch in C major using predefined dictionaries for ClefG and ClefF, as shown in Tab. 1.



Figure 5. Staff lines detection

| Note                                                                                | -1 | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7   | 8   | 9   |
|-------------------------------------------------------------------------------------|----|----|----|----|----|----|----|----|-----|-----|-----|
| C                                                                                   | 0  | 12 | 24 | 36 | 48 | 60 | 72 | 84 | 96  | 108 | 120 |
| C#                                                                                  | 1  | 13 | 25 | 37 | 49 | 61 | 73 | 85 | 97  | 109 | 121 |
| D                                                                                   | 2  | 14 | 26 | 38 | 50 | 62 | 74 | 86 | 98  | 110 | 122 |
| D#                                                                                  | 3  | 15 | 27 | 39 | 51 | 63 | 75 | 87 | 99  | 111 | 123 |
| E                                                                                   | 4  | 16 | 28 | 40 | 52 | 64 | 76 | 88 | 100 | 112 | 124 |
| F                                                                                   | 5  | 17 | 29 | 41 | 53 | 65 | 77 | 89 | 101 | 113 | 125 |
| F#                                                                                  | 6  | 18 | 30 | 42 | 54 | 66 | 78 | 90 | 102 | 114 | 126 |
| G                                                                                   | 7  | 19 | 31 | 43 | 55 | 67 | 79 | 91 | 103 | 115 | 127 |
| G#                                                                                  | 8  | 20 | 32 | 44 | 56 | 68 | 80 | 92 | 104 | 116 |     |
| A                                                                                   | 9  | 21 | 33 | 45 | 57 | 69 | 81 | 93 | 105 | 117 |     |
| A#                                                                                  | 10 | 22 | 34 | 46 | 58 | 70 | 82 | 94 | 106 | 118 |     |
|  | 11 | 23 | 35 | 47 | 59 | 71 | 83 | 95 | 107 | 119 |     |

Figure 6. MIDI note to pitch correspondence

| Unit Distance | Pitch (ClefG) | Pitch (ClefF) |
|---------------|---------------|---------------|
| 6             | 81 (A5)       | 60 (C4)       |
| 5             | 79 (G5)       | 59 (B3)       |
| 4             | 77 (F5)       | 57 (A3)       |
| 3             | 76 (E5)       | 55 (G3)       |
| 2             | 74 (D5)       | 53 (F3)       |
| 1             | 72 (C5)       | 52 (E3)       |
| 0             | 71 (B4)       | 50 (D3)       |
| -1            | 69 (A4)       | 48 (C3)       |
| -2            | 67 (G4)       | 47 (B2)       |
| -3            | 65 (F4)       | 45 (A2)       |
| -4            | 64 (E4)       | 43 (G2)       |
| -5            | 62 (D4)       | 41 (F2)       |
| -6            | 60 (C4)       | 40 (E2)       |

Table 1. Pitch and unit distance mapping in C major

**Temporal Grouping.** All musical elements are sorted from left to right and grouped by horizontal proximity. Elements in the same group are assumed to occur simultaneously (e.g., chords or stacked notes). Chords are assigned the same onset time.

**MIDI Encoding.** Each note is translated into a pair of `note_on` and `note_off` MIDI messages. Chords are encoded by sending all `note_on` messages with the same timestamp, followed by `note_off` messages after the computed duration. Rests are treated as silent time intervals and accumulate delay for subsequent notes.

This purely rule-based system ensures deterministic pitch and timing assignments from bounding box coordinates, enabling symbolic reconstruction of the sheet music.

## 4. Results

Our cropping-based strategy effectively reduces contextual complexity while preserving the musically relevant structure of sheet music. This approach led to a notable performance improvement, achieving an overall mAP@50 of 0.865. We further visualized and compared our detection results against DeepScores’ baseline method (DWD) on the test set. As demonstrated in our experiments, our method significantly enhances detection accuracy—particularly for notehead-related classes—addressing a key limitation of full-page detection frameworks.

Fig. 7 Our pairwise detection approach consistently outperforms DWD and YOLO direct prediction.

| AP@0.5 Accuracy for 3 different OMR function, sorted by DWD highest accuracy |        |        |                   |
|------------------------------------------------------------------------------|--------|--------|-------------------|
| Class                                                                        | DWD    | YOLOv8 | YOLOv8 + pairwise |
| all                                                                          |        | 0.674  | 0.865             |
| rest16th                                                                     | 0.8773 | 0.916  | 0.994             |
| noteheadBlack                                                                | 0.8619 | 0.783  | 0.91              |
| keySharp                                                                     | 0.8185 | 0.917  | 0.988             |
| tuplet6                                                                      | 0.8028 | 0.73   | 0.982             |
| restQuarter                                                                  | 0.7942 | 0.91   | 0.983             |
| rest8th                                                                      | 0.7803 | 0.876  | 0.988             |
| noteheadHalf                                                                 | 0.7474 | 0.8315 | 0.979             |
| flag8thUp                                                                    | 0.7325 | 0.739  | 0.965             |
| flag8thDown                                                                  | 0.6634 | 0.781  | 0.98              |
| accidentalSharp                                                              | 0.6626 | 0.861  | 0.983             |
| accidentalNatural                                                            | 0.6559 | 0.681  | 0.947             |
| tuplet3                                                                      | 0.6298 | 0.533  | 0.938             |
| noteheadWhole                                                                | 0.6265 | 0.89   | 0.9845            |
| dynamicMF                                                                    | 0.5563 | 0.795  | 0.953             |
| rest32nd                                                                     | 0.542  | 0.947  | 0.995             |
| flag16thUp                                                                   | 0.532  | 0.766  | 0.978             |
| restWhole                                                                    | 0.518  | 0.639  | 0.763             |
| timeSig8                                                                     | 0.518  | 0.855  | 0.961             |
| accidentalFlat                                                               | 0.4949 | 0.772  | 0.972             |
| keyFlat                                                                      | 0.4685 | 0.874  | 0.985             |

Figure 7. Comparison of Our Method vs. DWD on Top 20 Most Accurate Classes:

Given the lack of reliable sheet-to-MIDI conversion tools, we adopted a hybrid validation approach: first converting our MIDI output back to sheet music using LilyPond, then manually comparing it to the original score. While automated machine comparison would have been ideal, we found it unreliable due to inherent ambiguities in music notation—such as different symbols representing the same musical meaning (e.g., grace notes vs. appoggiaturas) or slight rhythmic



Top: Original Music Sheet



Bottom: Generated Music Sheet

Figure 8. Comparison of the original and generated scores.

misalignments introduced during conversion. Our analysis confirmed that the model achieved near-perfect accuracy for simple C major scores, demonstrating robust performance on straightforward transcriptions, as shown in Fig. 8.

## 5. Conclusion

In this project, we presented **Staff2Sound**, an end-to-end deep learning pipeline that bridges the gap between visual sheet music and auditory MIDI output. By combining YOLOv8-based symbol detection with a pairwise staffline cropping strategy, we significantly improved note detection accuracy in dense musical layouts. Our rule-based pitch reconstruction module further enabled precise conversion of detected symbols into playable MIDI files, validated through a hybrid approach using LilyPond and manual inspection.

While the system excels at simple C major scores, challenges remain with complex notations. Our work provides a strong foundation for scaling toward full-page and polyphonic MIDI generation.

## References

- [1] Michael Scott Asato Cuthbert. Music21. <https://github.com/cuthbertLab/music21>, 2025. 2
- [2] Joseph Redmon, Santosh Kumar Divvala, Ross B. Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. *CoRR*, abs/1506.02640, 2015. 2
- [3] LilyPond Development Team. Lilypond. <https://github.com/lilypond/lilypond>, 2025. 2
- [4] MuseScore Development Team. Musescore 4.4.3. <https://musescore.org/en>, 2025. 2
- [5] Lukas Tuggener, Ismail Elezi, Jürgen Schmidhuber, Marcello Pelillo, and Thilo Stadelmann. Deepscores - A dataset for segmentation, detection and classification of tiny objects. *CoRR*, abs/1804.00525, 2018. 1

- [6] Lukas Tuggener, Ismail Elezi, Jurgen Schmidhuber, and Thilo Stadelmann. Deep watershed detector for music object recognition, 2018. 2

## 6. Critique Response

We used the DeepScores dataset which consists mainly of European classical music (e.g., Bach, Beethoven, Mozart), reinforcing a bias towards Western art music. This excludes many non-Western musical traditions (e.g., Indian ragas, Gamelan, or Chinese Jianpu notation) that use entirely different symbolic systems. Also, our output is set to be a MIDI file. MIDI defaults to 12-tone equal temperament, which is standard in Western music. However, many cultures use microtonal scales, which MIDI struggles to represent without customization. Moreover, to convert sheets to MIDI, we manually assigned the meters to MIDI rhythm. We are fully aware that the Western notation prioritizes divisible meters (4/4, 3/4, etc.), while many cultures use additive rhythms like Balkan music in 7/8 or free-flowing rhythms. Our system might misinterpret or oversimplify these structures. To mitigate the bias in this project, we could expand the training dataset to include global music traditions. We would certainly wish our project to not marginalize non western notations. Ideally, our tool could be adapted to transcribe non-notated music like field recordings into MIDI. This would broaden its cultural impact.

Our underlying object detection method in sheet music recognition is YOLO. YOLO is great for detecting discrete symbols like notes, rests, clefs in Western staff notation and handling crowded scores. But it can introduce OCR-related limitations. Through out the project, we found that YOLO is not a full OCR pipeline—it detects symbols but doesn't inherently understand context like a "#" could mean a sharp or a cautionary accidental. To fix the contextual issue, we manually implemented grammar rules to fix misclassifications. Also, YOLO doesn't know how to parse relationships, for instance, a note's duration depends on its stem/beam grouping which YOLO cannot detect. Once again, we fix the time duration by manually assign the beats to each note.

While our model, trained on the DeepScore dataset and used YOLO for symbol detection, demonstrates strong performance for Western classical music, we recognize its limitations in handling non-Western notations. To address these biases, we have documented the system's constraints and propose future enhancements, such as incorporating diverse musical datasets for global traditions. We also emphasize the importance of user feedback in identifying errors and improving accuracy.

User-agency is also important to us. While our system provides a fast and structured conversion from sheet music to MIDI, we realize that the output should serve as a flexible starting point rather than a definitive performance. Thus, we provide editable MIDI outputs so musicians can refine

automated interpretations.

Lastly, we want to highlight Ethical Openness in our project. We commit to transparency by openly documenting our dataset sources, model limitations, and potential biases, allowing users to understand the system's constraints and make informed decisions. By avoiding copyrighted training data, we mitigate legal risks and promote equitable access to music digitization tools.

By openly acknowledging these challenges and prioritizing inclusivity in future iterations, we aim to build a tool that not only serves a broader range of musical practices but also upholds fairness and accountability in music technology.



## Appendix

### Team contributions

Please describe in one paragraph per team member what each of you contributed to the project.

**Yuehuan Qiu** I was primarily responsible for the development and implementation of the rule-based pitch reconstruction module, including pitch inference, tonal mapping, staff line quantization, and MIDI track construction. I devised the method for computing the unit distance from each note to the center line of its associated staff, which is essential for converting vertical positions into discrete *staff position indices*. Building on this, I implemented the logic for clef-specific pitch mapping, using predefined dictionaries tailored for G-clef and F-clef in the C major scale. To make this mapping interpretable and reusable, I designed a reference table (Tab. 1) and verified its correctness across multiple staff configurations. In addition, I led the research and coding effort for generating MIDI files from the annotated musical symbols. This involved grouping note elements into temporal events, estimating note durations based on beams and flags, and encoding them as MIDI `note_on` and `note_off` events.

**Yifan Sun** I was also responsible for the development of the rule-based pitch reconstruction module. Together with Yuehuan, we conducted research on MIDI file structure and event encoding to ensure accurate musical translation. I implemented the logic for note duration estimation, including support for whole, half, and quarter notes. Additionally, I proposed incorporating beams and flags into the duration inference process—an idea that was later adopted into our final pipeline to improve rhythmic accuracy. I also designed and implemented a MIDI visualizer that converts MIDI output into a matplotlib plot using Python. This tool significantly aided our debugging process by allowing us to visually inspect pitch and timing across the generated notes. It was especially useful when fine-tuning the threshold values for flag and beam detection, ensuring that duration assignments were both musically and visually consistent.

**Wanming He** I generated the final sheet music output by converting MIDI files into notation using LilyPond, ensuring accurate and visually polished results. Additionally, I explored the integration of the SMT (Symbolic Music Transformer) model—a transformer-based architecture designed for music generation and transcription—to enhance our system’s capabilities. While the model performed well on reconstructing its training data, it struggled with generalization on unseen scores,

limiting its practical application in our pipeline. We eventually quit this idea and use the YOLO approach. Beyond technical implementation, I consolidated our team’s research, experiments, and findings into a poster for presentation.

**Peiyuan Li** I was responsible for the end-to-end preparation of the DeepScores dataset, including data cleaning and format conversion to make it compatible with YOLO training. I fine-tuned the YOLOv8 model specifically for symbol detection on the DeepScoresV2 Dense dataset. Additionally, I contribute the pair-wise detection strategy. I organized and refactored the codebase on GitHub, adding detailed annotations and creating a comprehensive README to help users understand and reproduce the project. For the written report, I authored the **Abstract**, **Introduction**, and the YOLO-related section of the **Methodology**. During the final presentation, I improved the clarity and structure of the system overview diagram in our poster to better convey the pipeline logic.